1 Лекция 14. Параметризованные редукции и W[1]-трудность

1.1 Зачем нужны редукции с контролем параметра

Мы уже видели задачи с алгоритмами $2^k N^{O(1)}$ и задачи, для которых простой перебор даёт $N^{O(k)}$. Из неудачи придумать алгоритм нельзя заключить его невозможность. **Редукция (reduction)** даёт проверяемый способ сравнить две задачи: преобразовать вход первой во вход второй так, чтобы ответы совпали.

В обычной теории NP-полноты достаточно полиномиального времени преобразования. Здесь важно сохранить малость параметра. Если из задачи с параметром $k = 3$ получилось $k' = n - 3$, то FPT-решатель цели может занять экспоненциальное время от n. Поэтому параметризованная редукция отдельно ограничивает k' функцией только от k.

Базовая задача для этой главы — **Clique**: найти k попарно смежных вершин графа. Её стандартная сложностная классификация — W[1]-полнота. Мы дадим точное определение редукции и её свойств, подробно построим переход к Multicolored Clique, затем объясним, что означает класс W[1] и условный вывод о трудности. Термин «трудность» здесь не означает доказательство FPT $\neq W[1]$.

Обозначение $A \leq_{\mathrm{FPT}} B$ читается как «A сводится к B». Это позволяет решать A, имея решатель B. Поэтому верхняя оценка переносится от B к A, а свидетельство трудности — от A к B. Полезно всякий раз мысленно выписывать алгоритм: вход $A \to$ преобразование $\to$ решатель $B \to$ ответ для A.

1.2 Три условия редукции

FPT many-one reduction из A в B переводит (x, k) в (x', k') так, что: ответы эквивалентны; $k' \leq g(k)$ для вычислимой g; время не больше $f(k)N^c$ с постоянной c. В отличие от обычной полиномиальной редукции, отдельно контролируется параметр. Время может содержать произвольную вычислимую зависимость от него.

Если B решается за $h(k')|x'|^d$, то после редукции общее время ограничивается

$$f(k)N^c + h(g(k))(f(k)N^c)^d,$$

где h можно заменить монотонной мажорантой. Это FPT по k. Следовательно, трудность переносится **от источника к цели**: чтобы показать трудность B, сводим к ней уже трудную A. Редукция $B \to A$ сама по себе этого не доказывает.

Та же подстановка доказывает транзитивность редукций: параметры композируются как $g_2(g_1(k))$, размер промежуточного входа ограничивается временем его построения. Степень по N остаётся постоянной.

1.3 Clique и Multicolored Clique

В Multicolored Clique задано разбиение вершин на k цветовых классов $V_1, \ldots, V_k$. Нужно выбрать по одной вершине из каждого класса так, чтобы они образовали клику.

Из Clique (G, k) создаём k копий множества вершин: $(i, v) \in V_i$. Соединяем (i, u) и (j, v) в точности когда $i \neq j$, $u \neq v$ и $uv \in E(G)$. Внутри класса рёбер нет. Параметр остаётся k, вершин kn, рёбер $O(k^2n^2)$, время FPT.

Если в G есть клика из k вершин, распределяем её по классам и получаем colorful-клику. Обратно, рёбра между выбранными копиями гарантируют разные исходные вершины и все

попарные смежности. Значит, редукция корректна. Цвета здесь — часть входных ограничений; это не случайные цвета из color coding. Малое число цветов не делает проверку совместимости FPT автоматически.

В Independent Set переход $G \mapsto \overline{G}$ сохраняет размер решения k и эквивалентен Clique. Но стандартный переход Independent Set к Vertex Cover меняет бюджет на $n - k$, который не ограничен функцией k. Он полиномиален, но не является FPT-редукцией для этих параметризаций.

1.4 Почему Multicolored Clique действительно эквивалентна Clique

В обратную сторону достаточно удалить все рёбра внутри цветовых классов и забыть цвета. В k-частном графе клика размера k содержит не больше одной вершины каждого класса; поскольку классов ровно k, она содержит ровно одну из каждого. Параметр остаётся k, время полиномиально. Вместе с прямой конструкцией получаем взаимные FPT-редукции.

На примере треугольника a, b, c и $k = 3$ прямая конструкция создаёт три класса копий. Выбор $(1, a), (2, b), (3, c)$ допустим. Выбор $(1, a), (2, a), (3, c)$ запрещён: две копии одной исходной вершины не соединены. Именно условие $u \neq v$ исключает многократное использование одной вершины; без него редукция была бы неверна.

Два взаимно сводимых представления могут быть удобны для разных построений. Цветовые классы позволяют заранее назначить роли вершинам решения, но не устраняют требование всех попарных смежностей.

1.5 Что такое W[1]

Для общего определения рассмотрим Boolean circuits. Входное присваивание имеет вес k, если ровно k входных битов равны 1. Weighted Circuit Satisfiability спрашивает, существует ли удовлетворяющее присваивание веса ровно k. Большой вентиль имеет fan-in больше 2; weft схемы — максимальное число больших вентилей на пути от входа к выходу. Depth — максимальная длина такого пути.

$W[t]$ состоит из задач, FPT-сводимых к этой задаче на схемах weft не больше t и глубины не больше некоторой константы d, зависящей от задачи, но не от входа. Для целей курса достаточно понимать смысл ограничений и пользоваться стандартной базовой теоремой: Clique и Independent Set по размеру решения k — $W[1]$-полные. Доказательство полноты общего семейства схем здесь не приводится (у источника теорема 13.18 также дана без полного доказательства).

Принадлежность Independent Set к $W[1]$ видна непосредственно: формула

$$\bigwedge_{uv \in E} (\neg x_u \vee \neg x_v)$$

имеет один большой AND, остальные вентили малы, глубина постоянна. Присваивание веса k выбирает независимое множество из k вершин. Пустые и малые схемы обрабатываются отдельно.

1.6 Как читать результат о трудности

$W[1]$-трудность означает, что все задачи $W[1]$ сводятся к данной; достаточно построить редукцию из Clique. Это не безусловное доказательство отсутствия FPT: вывод условен на FPT $\neq W[1]$. Для полноты дополнительно нужно включение в $W[1]$. Один NP-hardness proof этого не заменяет.

Для ориентира Set Cover по числу выбранных множеств и Dominating Set по размеру решения — стандартные $W[2]$-полные задачи (результаты W-иерархии используются как внешние). Лекция 13 не противоречит этому: там параметр Set Cover — размер вселенной. Лекция 12 также решает Dominating Set по treewidth, а не по размеру доминирующего множества.

Самопроверка: какие три проверки нужны для каждой новой редукции? Что нужно добавить к доказательству трудности, чтобы сказать «полна»? Почему алгоритм Clique за n^k не опровергает её $W[1]$-трудность?

1.7 Практика

Условия задач 14.1–14.5. Решения открывайте после своей попытки.

1.8 Источники и дополнительное чтение

Письменный источник: Cygan et al., *Parameterized Algorithms*, авторская копия 30.05.2016: §§13.1–13.3, с. 424–429 и 435–439. PDF. Номера страниц печатные; номер страницы PDF больше на 16.

Изложение, поясняющие примеры и упражнения подготовлены для этого курса. Ссылки фиксируют происхождение результатов; для чтения конспекта видео не требуется. Необязательные видео собраны в программе.